ESTRUCTURAS DE DATOS - x

Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Día, fecha: xx xx xxxx
Hora de inicio: 17:20
Auxiliar: [por completar]
Semana: [por confirmar]

TABLAS HASH

Unidad 3

Tema: Tabla Hash

Definición, características y operaciones
Escuela de Ingeniería de Ciencias y Sistemas
Universidad de San Carlos de Guatemala

Anuncios

xxxx
xxxx
xxxx
xxxx

Sustituir estos campos cuando estén disponibles los avisos oficiales del curso. No se han supuesto fechas de entrega ni cambios de proyecto.

AGENDA

Dudas del proyecto
Definición y características
Funciones hash y colisiones
Operaciones fundamentales

Factor de carga y rehashing
Análisis de rendimiento
Ejemplo práctico
Dudas y cierre

Valor de la semana: Em

Definición

La empatía consiste en comprender y compartir los sentimientos, necesidades y perspectivas de las demás personas. En clase se demuestra mostrando apertura hacia las opiniones ajenas y ofreciendo apoyo mutuo dentro del entorno de aprendizaje.

Aplicación en el curso

En la práctica académica:

- •Validar las emociones y opiniones de los compañeros.
- Ofrecer apoyo a quienes enfrentan dificultades con un tema.
- Mantener una actitud paciente ante diferentes ritmos de aprendizaje.
- Evitar juicios apresurados o críticas destructivas.

Competencias a desarrollar

Comprensión e implementación

• Explicar cómo una clave se transforma en una posición de almacenamiento.
• Implementar inserción, búsqueda, actualización y eliminación.
• Resolver colisiones con estrategias coherentes.

Análisis y verificación

• Interpretar el factor de carga.
• Comparar costo esperado y peor caso.
• Elegir entre hashing y estructuras ordenadas.
• Verificar una implementación mediante trazas.

2.1 Contenido

No.	Unidad	Tema	Sub Tema
1	3	Tabla Hash	Definición y características de la tabla hash
2	3	Tabla Hash	Operaciones de una tabla hash
3	3	Tabla Hash	Funciones hash y cálculo de índices
4	3	Tabla Hash	Colisiones: encadenamiento y direccionamiento abierto
5	3	Tabla Hash	Factor de carga, redimensionamiento y rehashing
6	3	Tabla Hash	Análisis de complejidad y comparación de estructuras
7	3	Tabla Hash	Aplicaciones y ejercicios prácticos de implementación

TABLAS HASH

Acceso eficiente mediante claves

De la función de dispersión a la implementación de un diccionario

1. Definición de tabla hash

Fundamento

Una tabla hash es una estructura que guarda asociaciones clave-valor y utiliza una función de dispersión para decidir dónde almacenar o buscar cada asociación.

Sus componentes son una colección de posiciones, una función hash y una estrategia para resolver colisiones.

Ejemplo y aplicación

Clave: carné del estudiante.
Valor: nombre, carrera u otros datos.

Ejemplo: 202010298 → «Estudiante A».

La posición física depende de la capacidad y de la función empleada. La clave debe almacenarse para distinguir registros que lleguen al mismo índice.

2. El modelo de diccionario: clave y valor

Fundamento

Un diccionario permite consultar un valor mediante una clave única. La tabla hash es una forma de implementar ese modelo, pero no es la única: también pueden utilizarse árboles.

En este material se adopta la política de actualizar el valor cuando se inserta una clave ya existente.

Ejemplo y aplicación

put(clave, valor): inserta o actualiza.
get(clave): devuelve el valor o indica ausencia.
remove(clave): elimina la asociación.
contains(clave): comprueba existencia.

Un conjunto hash guarda únicamente claves. Un diccionario hash conserva asociaciones clave-valor.

3. Anatomía interna de la estructura

Fundamento

La capacidad m es el número de posiciones o cubetas. El tamaño n es la cantidad de asociaciones almacenadas.

En encadenamiento, una posición contiene una colección de entradas. En direccionamiento abierto, las entradas ocupan posiciones del arreglo y la búsqueda puede inspeccionar varias.

Ejemplo y aplicación

Ejemplo: m = 7 posiciones.
Índices válidos: 0, 1, 2, 3, 4, 5 y 6.

Si se guardan 4 claves, n = 4 aunque algunas compartan cubeta.

No confundir «cantidad de posiciones» con «cantidad de elementos almacenados».

4. Características y límites

Fundamento

La estructura favorece búsquedas por igualdad y permite cambios dinámicos. Con una buena dispersión y carga controlada, sus operaciones pueden tener costo esperado constante.

El orden de iteración depende de la implementación. No debe suponerse que las claves aparecen ordenadas numéricamente.

Ejemplo y aplicación

Es apropiada para: comprobar si existe un carné, recuperar un producto por código o contar palabras.

No ofrece por sí sola búsquedas eficientes por intervalo, mínimo, máximo o siguiente clave. Para esas consultas suele convenir una estructura que mantenga el orden.

5. Comparación de estructuras de búsqueda

Estructura	Búsqueda exacta	Mantiene orden	Uso frecuente
Lista sin ordenar	O(n)	No por clave	Colecciones pequeñas
Arreglo ordenado	O(log n)	Sí	Datos con pocos cambios
Árbol balanceado	O(log n)	Sí	Rangos y orden
Tabla hash	O(1) esperado*	No necesariamente	Acceso por igualdad

* Con dispersión adecuada y carga controlada. Peor caso general: O(n). No incluye el costo de procesar claves largas.

6. Función hash e índice de almacenamiento

Fundamento

Una función hash transforma una clave en un código. Luego ese código se reduce al intervalo válido del arreglo.

Representación conceptual:
índice = reducir(hash(clave), m).

Para ejemplos con enteros no negativos se usará directamente h(k) = k mod m.

Ejemplo y aplicación

Si m = 7 y k = 31:
h(31) = 31 mod 7 = 3.

El registro se dirige inicialmente a la posición 3. Si otra clave también genera 3, se aplica la estrategia de colisión. El índice no identifica de forma única a la clave.

7. Propiedades de una buena dispersión

Fundamento

Consistencia: una misma clave debe producir el mismo resultado mientras no cambien la función y sus parámetros.

Distribución: las claves deben repartirse de forma razonablemente uniforme.

Eficiencia: calcular el hash no debe costar más que el beneficio obtenido.

Ejemplo y aplicación

Contrato esencial:
Si a y b son claves iguales, deben tener el mismo hash.

La afirmación inversa no se cumple: dos claves diferentes pueden compartir hash o índice. Por eso siempre se compara la clave real durante una consulta.

8. Método de división: ejemplo completo

Fundamento

El método de división usa el residuo de dividir la clave entre la capacidad.

$h(k) = k \bmod m$

Es sencillo para ejercicios numéricos. Elegir m exige considerar los patrones de las claves; utilizar un número primo puede ayudar, pero no garantiza por sí solo una buena distribución.

Ejemplo y aplicación

Con m = 7:
10 mod 7 = 3
17 mod 7 = 3
24 mod 7 = 3
31 mod 7 = 3

La secuencia muestra una limitación: claves separadas por 7 colisionan aunque la capacidad sea prima.

9. Método de multiplicación

Fundamento

Una alternativa multiplica la clave por una constante A, con 0 < A < 1, y utiliza la parte fraccionaria para obtener un índice.

$h(k) = \text{floor}(m \times \text{frac}(k \times A))$

$\text{frac}(x) = x - \text{floor}(x)$. La calidad de la distribución depende de la elección de A y de la implementación.

Ejemplo y aplicación

Ejemplo didáctico: m = 10, A = 0.6 y k = 7.
$k \times A = 4.2$
$\text{frac}(4.2) = 0.2$
$h(7) = \text{floor}(10 \times 0.2) = 2$.

A = 0.6 se usa solo para simplificar la cuenta; no es una recomendación de implementación.

10. Hashing de cadenas de texto

Fundamento

Una cadena contiene varios símbolos, por lo que es necesario combinar sus códigos. Un esquema polinómico procesa cada carácter en orden y acumula el resultado.

$r = (b \times r + \text{código}(c)) \bmod m$

Se inicia con r = 0 y se repite para todos los caracteres.

Ejemplo y aplicación

Ejemplo: «AB», b = 31 y m = 7.
Código de A = 65; código de B = 66.
r1 = 65 mod 7 = 2.
r2 = (31 × 2 + 66) mod 7 = 2.

La consulta debe usar la misma codificación y las mismas reglas para distinguir mayúsculas y minúsculas.

11. Claves negativas, compuestas y mutables

Fundamento

Para ejercicios con residuos negativos puede normalizarse el índice como ((k mod m) + m) mod m.

Una clave compuesta combina campos de forma consistente con la igualdad: por ejemplo, código de curso y sección. No basta con concatenar datos sin definir sus límites.

Ejemplo y aplicación

Una clave no debe cambiar los campos usados por hash e igualdad mientras está guardada. Si cambia, el registro puede quedar en una posición distinta de la que buscaría la función.

Para cambiar una clave: retirar la asociación y volver a insertarla con la nueva clave.

12. Qué es una colisión

Fundamento

Una colisión ocurre cuando dos claves diferentes se asignan al mismo índice de la tabla. No significa que los registros sean iguales ni que exista un error de programación.

Si el universo de claves supera las posiciones disponibles, no es posible asignar un índice diferente a todas las claves posibles.

Ejemplo y aplicación

Con m = 7:
h(10) = 3
h(17) = 3

Ambas claves son válidas. La estructura necesita conservarlas y recuperar cada una sin confundirlas. El rendimiento depende tanto de la dispersión como del manejo de estas coincidencias.

13. Estrategias para resolver colisiones

Fundamento

Encadenamiento separado: cada índice conduce a una colección de entradas que comparten cubeta.

Direccionamiento abierto: cada entrada ocupa una posición del arreglo. Si el índice inicial está ocupado, se exploran otras posiciones mediante una secuencia de sondeo.

Ejemplo y aplicación

Variantes que estudiaremos:
- Encadenamiento.
- Sondeo lineal.
- Sondeo cuadrático.
- Doble hashing.

Para evitar ambigüedad entre términos de distintos textos, se utilizarán estos nombres explícitos durante toda la presentación.

14. Encadenamiento separado

Fundamento

Cada cubeta mantiene una lista u otra colección de pares clave-valor. Para buscar se calcula el índice y se recorre únicamente la colección correspondiente.

Las colisiones aumentan la longitud de la colección. La tabla puede almacenar más claves que cubetas, aunque el costo de recorrerlas aumenta.

Ejemplo y aplicación

m = 7; h(k) = k mod 7.

Cubeta 3: (10,A), (17,B), (24,C).
Cubeta 4: (11,D).

Buscar 17 requiere revisar la cubeta 3 y comparar las claves de sus entradas. No es suficiente devolver el primer elemento de esa cubeta.

15. Inserción y actualización con encadenamiento

Fundamento

1. Calcular la cubeta correspondiente.
2. Buscar la clave en esa colección.
3. Si existe, reemplazar su valor.
4. Si no existe, agregar una entrada e incrementar n.

Comprobar duplicados evita que el diccionario guarde dos valores independientes para una misma clave.

Ejemplo y aplicación

Estado inicial: [(10,A), (17,B)].
put(17, «Nuevo») cambia B por «Nuevo».
El tamaño n se mantiene.

put(24, C) agrega una asociación.
El tamaño n aumenta en uno.

La posición de inserción dentro de la lista depende de la política elegida.

16. Búsqueda y eliminación con encadenamiento

Fundamento

Búsqueda: calcular el índice y comparar la clave solicitada con las entradas de la cubeta. Si no aparece, informar ausencia.

Eliminación: localizar la clave, retirar su entrada y disminuir n. Si se usa una lista enlazada, deben actualizarse los enlaces que correspondan.

Ejemplo y aplicación

Antes: cubeta 3 = [10, 17, 24].
Buscar 17: encontrado.
Eliminar 17: cubeta 3 = [10, 24].
Buscar 17 otra vez: ausente.

Eliminar una clave inexistente no cambia el tamaño ni debe retirar una entrada distinta.

17. Direccionamiento abierto

Fundamento

En este enfoque todas las entradas residen en el arreglo principal. Una secuencia de sondeo determina qué posiciones examinar después de una colisión.

La inserción y la búsqueda deben seguir exactamente la misma secuencia para una clave determinada.

Ejemplo y aplicación

Estados posibles de una posición:
• VACÍO: nunca ocupada desde la reconstrucción.
• OCUPADO: contiene una asociación activa.
• BORRADO: antes contenía una entrada.

La diferencia entre VACÍO y BORRADO es fundamental para que las consultas sigan encontrando claves desplazadas.

18. Sondeo lineal: regla de exploración

Fundamento

El sondeo lineal avanza una posición cada vez y regresa al inicio al alcanzar el final del arreglo.

$h_i(k) = (h(k) + i) \bmod m$

i toma los valores 0, 1, …, m−1. Es necesario limitar el recorrido para evitar ciclos infinitos cuando no hay espacio.

Ejemplo y aplicación

Con m = 7 y h(k) = 3, la secuencia es: 3, 4, 5, 6, 0, 1, 2.

El arreglo contiguo favorece la localidad de memoria. Su principal limitación es la formación de grupos de posiciones ocupadas que alargan las consultas.

19. Traza de inserción con sondeo lineal

Fundamento

Insertar 10, 17, 24 y 31 en ese orden.
Capacidad m = 7; h(k) = k mod 7.
Todas las claves comienzan en el índice 3.

10 ocupa 3; 17 encuentra 3 ocupado y usa 4; 24 usa 5; 31 usa 6.

Ejemplo y aplicación

```
Estado del arreglo:
Índice:  0   1   2   3    4    5    6
Clave:   -   -   -  10   17   24   31

Sondeos por inserción: 1, 2, 3 y 4.
Total: 10 inspecciones de posiciones.

La colisión no sobrescribe una clave
diferente.
```

20. Búsqueda exitosa y búsqueda fallida

Fundamento

Una consulta empieza en h(k), compara la clave y sigue la secuencia hasta encontrarla o alcanzar una posición VACÍA.

Una posición BORRADA no detiene la búsqueda. Si todas las posiciones son inspeccionadas, la consulta termina indicando ausencia.

Ejemplo y aplicación

En la tabla anterior, buscar 24 revisa:
3 → 10; 4 → 17; 5 → 24.
Resultado: encontrado en 3 sondeos.

Buscar 38 comienza también en 3 y revisa 3, 4, 5, 6 y 0. Como 0 está VACÍO, confirma ausencia en 5 sondeos.

21. El problema de la agrupación primaria

Fundamento

El sondeo lineal puede formar bloques consecutivos de celdas ocupadas. Una nueva clave que llegue a cualquier punto del bloque tendrá que recorrerlo hasta alcanzar espacio.

Ese crecimiento se denomina agrupación primaria. La carga alta suele agravar su impacto.

Ejemplo y aplicación

Bloque ocupado: índices 3, 4, 5 y 6.

Una clave cuyo índice inicial sea 4 también recorre el bloque, aunque no comparta hash inicial con las que llegaron a 3.

La carga baja, una función adecuada y el redimensionamiento ayudan a controlar este efecto.

22. Sondeo cuadrático

Fundamento

El desplazamiento crece como una expresión cuadrática del número de intento.

$h_i(k) = (h(k) + c1 \cdot i + c2 \cdot i^2) \bmod m$

Reduce la agrupación primaria, pero las claves con igual índice inicial pueden seguir la misma secuencia, produciendo agrupación secundaria.

Ejemplo y aplicación

Ejemplo simplificado: $h_i = (h + i^2) \bmod 7$.
Con $h = 3$ e $i = 0, 1, 2, 3$:
3, 4, 0, 5.

La secuencia no garantiza visitar todas las celdas. Deben elegirse capacidad, constantes y límite de carga compatibles; tener celdas libres no siempre basta.

23. Doble hashing

Fundamento

Utiliza una segunda función para calcular el tamaño del salto.

$h_i(k) = (h1(k) + i \cdot h2(k)) \bmod m$

El salto debe ser distinto de cero y coprimo con m para recorrer todas las posiciones. Con m primo puede elegirse h2 entre 1 y m−1.

Ejemplo y aplicación

Ejemplo: m = 7, h1(k) = k mod 7 y h2(k) = 1 + (k mod 6).

Para k = 17: h1 = 3; h2 = 6.
Secuencia: 3, 2, 1, 0, 6, 5, 4.

Dos claves que coinciden en h1 pueden tener diferentes saltos y seguir rutas distintas.

24. Comparativa de estrategias de colisión

Estrategia	Dónde guarda	Ventaja	Cuidado principal
Encadenamiento	Colecciones por cubeta	Borrado sencillo	Enlaces y cadenas largas
Sondeo lineal	Arreglo principal	Buena localidad	Agrupación primaria
Sondeo cuadrático	Arreglo principal	Dispersa los saltos	Cobertura de posiciones
Doble hashing	Arreglo principal	Secuencias variadas	Salto coprimo con m

La elección depende del patrón de claves, carga, memoria y frecuencia de inserciones y eliminaciones.

25. Eliminación correcta y marcas de borrado

Fundamento

Si se elimina una entrada desplazada marcando su celda como VACÍA, una búsqueda posterior puede detenerse demasiado pronto.

Una solución consiste en marcar BORRADO y conservar la continuidad del sondeo. Existen alternativas, como reinserción del grupo, según la implementación.

Ejemplo y aplicación

Partimos de 3:10, 4:17, 5:24, 6:31.
Eliminar 17 deja 4:BORRADO.

Buscar 24 revisa 3, continúa por 4 y llega a 5.

Si 4 se hubiera marcado VACÍO, el algoritmo concluiría de forma incorrecta que 24 no existe.

26. Reutilización de posiciones borradas

Fundamento

Al insertar se recuerda la primera posición BORRADA, pero se continúa buscando por si la clave ya existe más adelante.

Solo después de confirmar la ausencia se utiliza la marca guardada o una posición VACÍA. Esta regla evita crear duplicados durante una actualización.

Ejemplo y aplicación

Estado: 3:10, 4:BORRADO, 5:24, 6:31.

put(24, «Nuevo»): actualiza la entrada en 5; no crea otra en 4.

put(38, E): puede reutilizar 4 tras confirmar que 38 no existe. La nueva clave conserva una ruta de consulta válida.

27. Factor de carga

Fundamento

El factor de carga relaciona los elementos activos con la capacidad.

alfa = n / m

En encadenamiento puede superar 1, pues cada cubeta admite varias entradas. En direccionamiento abierto n no puede superar m y conviene mantener espacio libre para las operaciones.

Ejemplo y aplicación

Si m = 7 y n = 4:
alfa = 4/7 ≈ 0.571.

Con tres entradas borradas, la carga activa no describe todo el trabajo: las marcas siguen afectando las búsquedas. Puede vigilarse también (n + borrados) / m para decidir cuándo reconstruir.

28. Umbrales y política de crecimiento

Fundamento

Una tabla dinámica define cuándo aumentar su capacidad o reconstruirse. El umbral depende de la estrategia y de los objetivos de memoria y velocidad.

No existe un porcentaje universal que sea óptimo para todas las tablas hash. Es necesario documentar la política adoptada.

Ejemplo y aplicación

Política didáctica para sondeo lineal: Ampliar antes de que la siguiente inserción deje alfa > 0.70.

Con m = 7 y n = 4, insertar una quinta clave daría 5/7 ≈ 0.714. Antes de insertarla, se crea una tabla mayor y se reubican las entradas.

29. Redimensionamiento y rehashing

Fundamento

Cambiar m modifica el resultado de la reducción del hash. Por eso no basta con copiar cada entrada al mismo índice de un arreglo mayor.

Rehashing: crear otra tabla, recorrer las entradas activas y volver a insertarlas usando la nueva capacidad y sus reglas de sondeo.

Ejemplo y aplicación

Ejemplo: pasar de m = 7 a m = 17.

Clave 10: de 3 pasa a 10.
Clave 17: de 3 pasa a 0.
Clave 24: de 3 pasa a 7.
Clave 31: de 3 pasa a 14.

Las marcas BORRADO no se trasladan: solo se reinsertan las asociaciones activas.

30. Costo esperado y costo amortizado

Fundamento

Con dispersión adecuada y carga acotada, una operación puede examinar una cantidad esperada constante de posiciones o entradas.

Una reconstrucción cuesta O(n) al mover las entradas activas. No se debe describir esa inserción concreta como si tuviera costo constante.

Ejemplo y aplicación

Si la capacidad crece geométricamente y se controla la carga, el trabajo de las reconstrucciones se reparte entre muchas inserciones.

Así se obtiene costo esperado amortizado O(1) por inserción bajo los supuestos del análisis. «Esperado» y «amortizado» describen aspectos diferentes.

31. Peor caso y complejidad de las claves

Fundamento

Una mala distribución puede concentrar muchas claves en una cubeta o generar largas secuencias de sondeo. En esas condiciones, una operación puede costar O(n).

Además, calcular el hash de una cadena de longitud L puede requerir O(L); comparar claves también puede tener un costo variable.

Ejemplo y aplicación

Las tablas no garantizan O(1) en todos los casos.

Al justificar el rendimiento se debe indicar:
- Cómo se distribuyen las claves.
- Qué carga se permite.
- Si hay reconstrucción.
- Qué cuesta procesar cada clave.

Para claves enteras de tamaño fijo, suele modelarse ese último costo como constante.

32. Resumen de complejidades

Operación	Esperado*	Peor caso general	Observación
Buscar / consultar	O(1)	O(n)	Sin orden por clave
Insertar / actualizar	O(1) amortizado	O(n)	Puede reconstruir
Eliminar	O(1)	O(n)	Gestionar colisiones
Recorrer tabla	O(m + n)	O(m + n)	Inspecciona posiciones
Rehashing	O(n) esperado**	Depende de reinserción	Reserva arreglo nuevo

* Buena dispersión, carga controlada y claves de costo fijo. ** Con nueva capacidad proporcional a n y reinserciones esperadas O(1). Recorrido: implementación convencional.

33. Pseudocódigo de inserción con encadenamiento

Fundamento

La comprobación de igualdad forma parte de la operación. Una coincidencia de índice no permite actualizar sin comparar la clave.

El contador n aumenta únicamente cuando se crea una asociación nueva. El crecimiento puede comprobarse antes de insertar, según el umbral de la implementación.

Pseudocódigo

```
put(k, v):
  i = indice(k, m)
  para entrada en cubetas[i]:
    si entrada.clave == k:
      entrada.valor = v
      retornar ACTUALIZADO
  cubetas[i].agregar((k, v))
  n = n + 1
  retornar INSERTADO
```

34. Pseudocódigo de búsqueda con sondeo lineal

Fundamento

La consulta distingue tres estados. Una celda BORRADA se omite; una VACÍA permite confirmar ausencia; una OCUPADA requiere comparar la clave.

El límite de m intentos garantiza la terminación incluso si todas las posiciones están ocupadas o marcadas como borradas.

Pseudocódigo

```
get(k):
  para intento = 0 hasta m-1:
    i = (h(k) + intento) mod m
    si estado[i] == VACIO:
      retornar AUSENTE
    si estado[i] == OCUPADO:
      si clave[i] == k:
        retornar valor[i]
  retornar AUSENTE
```

35. Aplicaciones de las tablas hash

Fundamento

Diccionarios y conjuntos: recuperación por clave y detección de pertenencia.

Compiladores: tablas de símbolos para asociar nombres con información.

Procesamiento de texto: conteo de frecuencias y eliminación de duplicados. Las claves se normalizan según el objetivo del programa.

Ejemplo y aplicación

Ejemplo: contar «árbol hash árbol».

Tras la primera palabra: árbol = 1.
Tras la segunda: hash = 1.
Tras la tercera: árbol = 2.

Debe definirse si «Árbol» y «árbol» son iguales. La función hash y la comparación deben respetar la misma decisión.

36. Tabla hash, árbol B y hash criptográfico

Fundamento

Una tabla hash favorece acceso por igualdad; un árbol B organiza claves y favorece consultas ordenadas y acceso por páginas. Elegir depende de la operación requerida.

Una función para dispersión no necesariamente ofrece resistencia frente a ataques o manipulación intencional de entradas.

Ejemplo y aplicación

Buscar un producto por código: tabla hash.
Recuperar un intervalo ordenado: árbol B/B+ según el sistema.

El almacenamiento seguro de contraseñas requiere métodos específicos con sal y costo de trabajo. No se consigue aplicando una función hash de diccionario al texto de la contraseña.

37. Verificación y errores frecuentes

Fundamento

Verificar que el número de entradas activas coincida con n, que una actualización no cree duplicados y que todas las asociaciones sean recuperables tras un rehashing.

Probar colisiones deliberadas ayuda a comprobar el algoritmo: una secuencia sin colisiones no valida la lógica de resolución.

Ejemplo y aplicación

Casos recomendados:
- Tabla vacía y clave ausente.
- Claves 10, 17 y 24 con m = 7.
- Actualización de clave existente.
- Borrado dentro de un grupo.
- Reutilización de BORRADO.
- Recorrido circular y tabla sin VACÍOS.
- Reconstrucción tras crecer.

38. Ejercicio guiado: operaciones paso a paso

Fundamento

Usar m = 7, h(k) = k mod 7 y sondeo lineal.

1. Insertar 10, 17, 24 y 31.
2. Buscar 24 y contar sondeos.
3. Eliminar 17 usando BORRADO.
4. Insertar 38 reutilizando la marca.
5. Buscar 17 y justificar la ausencia.

Ejemplo y aplicación

Solución esperada:
Tras insertar: 3:10, 4:17, 5:24, 6:31.
Buscar 24: 3 sondeos.
Eliminar 17: 4:BORRADO.
Insertar 38: 4:38.
Buscar 17: 5 sondeos, incluido 0 VACÍO.

Tamaño final n = 4; alfa = 4/7. Se conservan las claves 10, 24, 31 y 38.

39. Ejercicio de rehashing y cierre del ejemplo

Fundamento

Continuar con las claves activas 10, 38, 24 y 31. Antes de insertar 45, aplicar el umbral didáctico de 0.70: cinco claves superarían ese valor en m = 7.

Crear una tabla de capacidad 17, reinsertar las cuatro claves y después insertar 45.

Ejemplo y aplicación

Distribución final, usando k mod 17:
10 → índice 10
38 → índice 4
24 → índice 7
31 → índice 14
45 → índice 11

No hay colisiones en esta traza.
Carga final: 5/17 ≈ 0.294.
Pregunta: ¿por qué copiar los índices anteriores sería incorrecto?

Conceptos clave aprendidos

Clave y valor
Función hash e índice
Colisión y secuencia de sondeo
Encadenamiento y direccionamiento abierto

Marcas de borrado
Factor de carga
Redimensionamiento y rehashing
Costo esperado, amortizado y peor caso

Referencias

Texto abierto

Pat Morin. Open Data Structures.
Capítulo 5: Hash Tables.
https://opendatastructures.org/newhtml/ods/latex2/hashing.html

Material universitario

Robert Sedgewick y Kevin Wayne.
Algorithms, 4th edition. Sección 3.4.
https://algs4.cs.princeton.edu/34hash/

Recursos y práctica

Consulta del material

Open Data Structures: algoritmos y análisis de encadenamiento y sondeo lineal.

Princeton: implementaciones de referencia para comparar políticas de inserción, borrado y redimensionamiento.

Trabajo posterior

Actividad: implementar una de las dos estrategias y registrar el número de comparaciones.

Comparar una secuencia dispersa con otra cuyas claves colisionen. Explicar cómo cambian los resultados al aumentar la capacidad.

PAUSA DE HIDRATACIÓN

DUDAS

Repaso final: ¿cuándo se detiene una búsqueda?, ¿para qué sirve BORRADO?, ¿qué cambia al aumentar m?